CMP-466: Machine Learning and Data Mining
Music Genre Classification using High-Level Audio Features

Colab: Group7_MLProject.ipynb

List of Authors:

Muhammed Hady - B00094926

Pranav Vivek - B00090976

Arya - B00097405

Abstract

A summary of your project where you mention the purpose of the work, problem description, the dataset used, the methods used, the key results obtained (for the best model), and a summarizing sentence.

1. Introduction

Music streaming platforms organize and recommend content at massive scale, which makes automatic music genre classification an important task for search, discovery, playlisting, and recommendation. In practice, genre prediction can be approached using (a) raw audio and signal-processing features (e.g., spectrograms/MFCCs) or (b) high-level audio descriptors and metadata provided by platforms such as Spotify (e.g., danceability, energy, loudness, valence, tempo). Our project focuses on the second setting: tabular, high-level audio features, which are attractive because they are widely available, computationally cheaper than raw-audio pipelines, and can support more interpretable modeling.

Prior research using Spotify-style datasets shows that classical supervised learning can be effective, with tree-based approaches often performing strongly. For example, Fiqri et al. (2025) compared Naïve Bayes, k-NN, Decision Trees, and Random Forests for Spotify genre classification and found Random Forest achieved the best accuracy (≈79.4%), closely followed by Decision Trees (≈79.3%). Similarly, recent work on Spotify Tracks data for popularity/hit prediction also finds decision-tree ensembles competitive and highlights the importance of using metrics beyond accuracy (precision/recall/F1) and addressing dataset issues like imbalance and leakage.

Despite these advances, existing solutions have limitations that motivate our work. First, many papers either focus on different targets (e.g., hit prediction rather than genre) or do not fully document preprocessing, feature encoding, and hyperparameters, which makes results difficult to reproduce and compare fairly. Second, multi-class genre classification at large label counts (e.g., 100+ genres) is challenging because genres overlap in audio characteristics, so reporting only overall accuracy can hide failure modes across specific genres.

Our contribution is a reproducible end-to-end pipeline for multi-class genre classification using the public Spotify Tracks Dataset (114,000 tracks; 114 balanced genres) and its high-level audio features. We (1) implement transparent preprocessing (feature selection, encoding of categorical/binary attributes, and standardization of continuous features), and (2) benchmark interpretable classical models used in the literature (e.g., k-NN and decision-tree-based methods) under consistent evaluation. This addresses common limitations in prior work by emphasizing clarity, comparability, and practicality for genre modeling using tabular Spotify audio descriptors.

2. Literature Review

Spotify Audio Traits and Popularity Modelling [2]

Pacut's literature review discusses work that models song popularity using Spotify audio traits and metadata via regression and ML.

Purpose

- Investigate how traits such as energy, acousticness, release year, and others affect song popularity on Spotify, using the Web API audio features.

Methods

- Web-API-based datasets (track_id, acousticness, danceability, energy, etc.).
- Regression models to estimate popularity scores from features, and correlation analyses.

Dataset

- Spotify Web API datasets with year and popularity labels; almost the same columns as our dataset but with explicit temporal coverage for multiple decades.

Performance metrics & values

- Coefficients, p-values, R-squared of regression models.
- Key reported patterns:
    - Release year has high positive correlation with popularity, reflecting algorithmic emphasis on recent listening.
    - Acousticness has a strong negative correlation with popularity in current data.
    - Energy is a moderate positive predictor but not sufficient alone to explain popularity.

Findings

- Confirms that audio traits and temporal information (year) interact; the effect of traits like energy and acousticness on popularity is different across decades.

Critical review

- These works do not learn a "decade classifier," but their regression coefficients and correlation results show that metadata carries substantial temporal signal.
- For our project, these results justify:
    - Including release-year derived labels (e.g., 1960s, 1970s, …) as targets.
    - Testing models that combine audio features + popularity + genre as input for decade prediction.

Spotify Between Decades: Hit or Miss Predictor [3]

Purpose

- Predict whether a song is a hit or miss and see if the relationship between genre and key audio features differs between the 2000s and 2010s.

Methods

- Binary classification (hit vs non-hit) per decade.
- Models: Random Forest and Neural Network (LVQ).

- 70/30 train–test split, feature importance to find key variables, then add genre dummies and test correlations and time-change with the Augmented Dickey–Fuller test.

Dataset (public & private)

- Public Kaggle dataset: "Spotify Hit Predictor (1960–2019)" based on Spotify Web API.
- Subsets: 2000s (5,872 tracks) and 2010s (6,398 tracks), balanced hit/non-hit labels.
- Genre manually added and dummy-coded for 700 tracks per decade.

Findings

- Most important features: instrumentalness, danceability, loudness, energy, duration_ms, acousticness.
- Genre plus these features show significant correlations for several genres in both decades.
- Time-series test suggests the genre–feature relationship changes between 2000s and 2010s, so the null of "no change" is rejected.

Critical review

- Pros: Clear decade focus, strong performance with Random Forest, and inclusion of genre enriches earlier hit-prediction work.
- Cons: Genre is only annotated for a small subset (700 tracks per decade) and may be biased; interpretation of ADF p-value is somewhat inconsistent; only two decades are studied, limiting temporal scope.

Predicting Music Popularity: A Machine Learning Approach Using Spotify Data [4]

This research investigates the use of regression and classification models to forecast song popularity based on intrinsic audio features. Using a large-scale Spotify dataset, it identifies the specific acoustic drivers that determine whether a track will become a hit.

- **Purpose:** To evaluate the effectiveness of machine learning algorithms in forecasting song popularity and to analyze the influence of various audio features on a track's success.
- **Methods:** The study compares three machine learning models: Random Forest Regressor (RF), Simple Linear Regression, and Gradient Boosting Machines (GBM).
    - **Data Split:** The dataset was divided into 80% for training and 20% for testing using the train-test-split function.
- **Dataset (Public):** A dataset sourced from Spotify and Kaggle containing 114,000 songs recorded over two decades.
- **Performance Metrics & Values:**
    - **Regression:** The Random Forest Regressor was the most effective, achieving an R2 score of 0.6137 and a Mean Squared Error (MSE) of 190.61.
    - **Classification:** The model achieved an accuracy of 0.76 across multiple classification scenarios, including when popularity was divided into two, three, and four parts.
- **Findings:**

    - Random Forest was preferred for its ability to capture complex, non-linear relationships within audio data.
    - Popularity is strongly influenced by danceability, energy, and loudness.
    - Additional crucial indicators identified include valence, acousticness, and track genre.
- **Critical Review:** While successful, the study relied on ensemble models (Random Forest/GBM) to compensate for the poor performance of simpler linear models. It identified a challenge in predicting songs at the extreme ends of the popularity spectrum, which the author attributed to missing cultural and marketing data. Most importantly, the paper did not implement dimensionality reduction (PCA) to resolve feature multicollinearity or document exhaustive hyperparameter tuning for its baseline models.

Music genre classification with parallel convolutional neural networks and capuchin search algorithm [5]

This paper proposes a high-performance ensemble architecture that utilizes parallel Convolutional Neural Networks to categorize music. It introduces the Capuchin Search Algorithm (CapSA) to automate the fine-tuning of hyperparameters, achieving near-state-of-the-art accuracy across multiple audio feature sets.

- **Purpose:** To develop a robust music classifier that enhances user experience in streaming services by quickly categorizing unlabeled music using integrated signal processing and deep learning.
- **Methods:** The study proposes a Parallel Convolutional Neural Network (PCNN) ensemble architecture comprising three independent CNN models.
    - Each model processes a distinct audio feature: Discrete Wavelet Transform (DWT), Mel-frequency Cepstral Coefficients (MFCC), and Short-Time Fourier Transform (STFT).
    - Hyperparameters (filter size, number of filters, pooling types) are optimized using the Capuchin Search Algorithm (CapSA), a nature-inspired meta-heuristic technique.
    - The final features from the last fully connected layers of these three networks are concatenated and fed into a SoftMax classifier.
- **Dataset (Public):** The researchers evaluated their method using two widely accepted public datasets:
    - **GTZAN:** 1,000 audio tracks (30 seconds each) across 10 musical genres (e.g., blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock).
    - **Extended-Ballroom:** 4,180 audio tracks (30 seconds each) categorized into 13 unique dance-related groups such as chacha, jive, quickstep, and tango.
- **Performance Metrics & Values:**
    - **Accuracy:** Achieved an average classification accuracy of 96.20% on the GTZAN dataset and 96.08% on the Extended-Ballroom dataset.

    - **Precision & Recall:** On the GTZAN dataset, the model achieved a precision of 96.21% and a recall of 96.20%. For the Extended-Ballroom set, it reported a precision of 90.32% and a recall of 96.17%.
    - **AUC:** The model achieved an Area Under the Curve (AUC) of 0.99 for both datasets.
- **Findings:**
    - The parallel architecture captures a wider range of features (spectral, temporal, and frequency details) than single-model approaches.
    - The inclusion of CapSA significantly outperformed traditional grid search by finding optimal architecture settings that reduced training errors.
    - STFT features were found to perform best individually, but combining all three yielded the highest discriminative capability.
- **Critical Review:** The primary strength of this paper is its rigorous approach to hyperparameter optimization and feature diversity, leading to near-state-of-the-art results. However, the proposed model has high computational demands, requiring significantly more time for inference compared to single-feature CNNs. While the authors characterize deep learning as "simple to use" for retrieval, this approach remains a "black box" that lacks the semantic interpretability of traditional tabular machine learning.

Classification of Music Genres using Feature Selection and Hyperparameter Tuning [6]

Focusing on the traditional side of machine learning, this study demonstrates how targeted feature engineering and parameter optimization can push standard algorithms to peak performance. It emphasizes reducing model complexity to save computational resources while maintaining high accuracy on large-scale tabular datasets.

- **Purpose:** To efficiently categorize songs into genres by evaluating how specific feature selection and hyperparameter tuning enhance traditional ML performance.
- **Methods:** The authors compared several supervised learning models, including Random Forest (RF), Extreme Gradient Boosting (XGBoost), K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and Logistic Regression
    - **Feature Engineering:** The process involved removing non-essential attributes like artist_name and instance_id, followed by Label Encoding for categorical data and StandardScaler for normalization.
    - **Feature Selection:** Using Random Forest importance scores, the researchers identified the three most critical features (Popularity, Acousticness, and Danceability) to optimize the model's complexity.
    - **Hyperparameter Tuning:** The study utilized Grid Search and Randomized Search to identify the optimal parameters for the Random Forest classifier.
- **Dataset (Public):** The study utilized a Kaggle dataset containing 50,005 songs. Each entry includes 11 musical traits, such as energy, instrumentalness, liveness, loudness, and valence.

- **Performance Metrics & Values:**
  - **Random Forest:** Achieved an accuracy of 99.60% with all features and 99.48% when reduced to the top 3 features.
  - **XGBoost:** Achieved an accuracy of 99.60%.
  - **KNN:** Reported an accuracy of 56.75%.
  - **SVM:** Reported an accuracy of 54.06%.
  - **Logistic Regression:** Reported an accuracy of 51.19%.
- **Findings:**
  - Reducing the model to just three key features significantly optimized time and space complexity without a meaningful loss in accuracy.
  - The model occasionally produced incorrect predictions for genres that exhibit highly similar acoustic profiles.
  - Random Forest was determined to be the most effective model when paired with hyperparameter tuning and targeted feature selection.
- **Critical Review:** A major strength is the explicit comparison of Memory and Training Time; reducing the features from 11 to 3 reduced training time from 14.95ms to just 0.16ms. Unlike deep learning approaches, this study provides clear insights into which specific musical characteristics (like popularity) drive classification. The near-perfect accuracy (99.6%) on a large Kaggle dataset may indicate high feature correlation or specific data patterns that favor ensemble tree models.

Implementation of Supervised Learning Algorithm on Spotify Dataset [7]

- **Purpose:** Compare classic supervised ML algorithms for Spotify music genre classification and identify the best-performing model.
- **Methods:** The authors evaluate Naïve Bayes, k-NN, Random Forest, and Decision Tree classifiers and compare performance using cross-validation (they report accuracy and Cohen's kappa from a confusion-matrix-based evaluation).
- **Dataset (public/private): Public Kaggle "Spotify song dataset"** with audio-feature attributes (e.g., danceability, energy, loudness, speechiness, acousticness, instrumentalness, valence, tempo, duration_ms).
- **Performance metrics & values:** Reported results (Table 2):
  - Random Forest: Accuracy = 79.40%, Kappa = 0.576
  - Decision Tree: Accuracy = 79.30%, Kappa = 0.575
  - Naïve Bayes: Accuracy = 77.28%, Kappa = 0.534
  - k-NN: Accuracy = 60.74%, Kappa = 0.195
- **Findings:** Tree-based methods perform best, with Random Forest slightly outperforming Decision Tree and both beating Naïve Bayes and k-NN.
- **Critical review (limitations).**

- The paper provides limited detail on the exact dataset version, number of genres/classes, preprocessing choices, and hyperparameters—making replication and fair comparison harder.
- The evaluation relies primarily on accuracy/kappa, without macro-F1 or per-class performance, which matters in multi-class genre prediction.
- The workflow seems focused on tool-based implementation (RapidMiner) rather than reporting modeling choices in depth, which can hide key drivers of performance (feature scaling, encoding decisions, tree depth, k choice, etc.).

Predicting Hit Songs Using Spotify Tracks Dataset [8]

- **Purpose:** Predict whether a track becomes a "hit" using only Spotify audio features, where "hit" is defined as tracks at/above the 80th percentile of popularity (binary classification).
- **Methods:** Trains and compares Logistic Regression, SVM, Random Forest, and a Neural Network. Uses stratified 5-fold cross-validation and explicitly addresses class imbalance (class weighting, undersampling, and decision-threshold tuning to emphasize recall).
- **Dataset (public/private).** Uses the public Kaggle Spotify Tracks Dataset (~114k songs; includes audio features like danceability/energy/loudness/valence/tempo and metadata including track_genre).
- **Performance metrics & values:** Reports mean ± std across CV folds (Table 1):
  - Logistic Regression: Accuracy 0.580 ± 0.003, Precision 0.569 ± 0.002, Recall 0.655 ± 0.008, F1 0.609 ± 0.005
  - SVM: Accuracy 0.633 ± 0.006, Precision 0.607 ± 0.004, Recall 0.752 ± 0.011, F1 0.672 ± 0.007
  - Random Forest: Accuracy 0.728 ± 0.006, Precision 0.717 ± 0.006, Recall 0.756 ± 0.011, F1 0.736 ± 0.006
  - Neural Network: Accuracy 0.638 ± 0.005, Precision 0.637 ± 0.012, Recall 0.642 ± 0.050, F1 0.638 ± 0.021
- **Findings:** Random Forest achieves the best overall balance (highest F1 with low variance), while SVM achieves very strong recall. The paper's EDA suggests hits differ systematically in features like danceability/energy/loudness.
- **Critical review (limitations):**
  - The "hit" label is derived from a time-varying popularity score, so results can drift over time and may not generalize out-of-sample temporally.
  - The author notes potential data leakage (songs by the same artist appearing across folds), which can inflate metrics if artist identity correlates with popularity.
  - While the evaluation is stronger than accuracy-only reporting (includes precision/recall/F1), this paper targets hit prediction, not multi-class genre prediction—so it informs our project mainly on tabular audio-feature modeling practices (imbalance, leakage awareness, metric selection).

Summary of Motivation and Objectives

Current literature proves that high-level audio features are effective for predicting musical trends. However, most studies are limited to small datasets with only 10 to 13 genres. Research often favors complex ensemble models or "black box" deep learning architectures that, while accurate, lack interpretability and require significant computational resources. Furthermore, many existing papers overlook the impact of feature redundancy and fail to document the specific preprocessing choices that are vital for model reproducibility.

These shortcomings motivate the following objectives for this project:

- **Evaluate High-Cardinality Classification:** Perform robust genre classification across a significantly larger label set (114 genres) to test model scalability.
- **Optimize Feature Efficiency:** Implement and evaluate dimensionality reduction techniques to address feature redundancy and mitigate the "curse of dimensionality."
- **Compare Traditional Model Architectures:** Contrast the performance of various traditional machine learning models to identify which logic better suits high-level musical metadata.
- **Refine via Systematic Tuning:** Optimize model parameters to maximize accuracy while maintaining the interpretability and transparency of the classification process.

3. Methods and Datasets

3.1. Dataset

Source and Type: The dataset is a public tabular collection sourced from Kaggle and Hugging Face, curated by Maharshi Pandya. It consists of high-level acoustic features and metadata for a wide array of musical tracks extracted via the Spotify Web API [1].

Size and Shape: The dataset contains 114,000 records and 21 columns, resulting in a dataset shape of (114000, 21).

Features: The 21 features represent a mix of metadata and audio descriptors:

- **Metadata:** track_id, artists, album_name, and track_name.
- **Numerical/Audio Features:** popularity, duration_ms, danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, and time_signature.
- **Boolean:** explicit.
- **Target Label:** track_genre.

Class Labels and Distribution: The target variable for this project is

track_genre. The dataset covers 114 unique musical genres. The class distribution is perfectly balanced, with 1,000 records assigned to each genre, ensuring that no single class biases the training process.

Descriptive statistics: Descriptive statistics for the dataset's continuous numerical features can be seen below:

Feature	Mean	Std Dev	Min	25%	50% (Median)	75%	Max
Popularity	33.24	22.31	0.00	17.00	35.00	50.00	100.00
Duration (ms)	228,029	107,298	0.00	174,066	212,906	261,506	5,237,295
Danceability	0.57	0.17	0.00	0.46	0.58	0.70	0.99
Energy	0.64	0.25	0.00	0.47	0.69	0.85	1.00
Loudness (dB)	-8.26	5.03	-49.53	-10.01	-7.00	-5.00	4.53
Speechiness	0.08	0.11	0.00	0.04	0.05	0.08	0.97
Acousticness	0.31	0.33	0.00	0.02	0.17	0.60	1.00
Instrumentalness	0.16	0.31	0.00	0.00	0.00	0.05	1.00
Liveness	0.21	0.19	0.00	0.10	0.13	0.27	1.00
Valence	0.47	0.26	0.00	0.26	0.46	0.68	1.00

Tempo (BPM)	122.15	29.98	0.00	99.22	122.02	140.07	243.37

3.2. **Missing Values:** Upon inspection, only one record contained missing values. Given that this represents a negligible portion of the 114,000-record dataset, this observation was removed to maintain data integrity.

Feature Selection: Metadata columns, including track_id, artists, album_name, and track_name, were dropped as they do not provide generalizable musical patterns. Additionally, the Unnamed: 0 column, which served as an index in the CSV file, was removed.

Binary Conversion: The mode and explicit features are binary by nature. These were represented as binary integers (0 and 1). Since mode was already provided in this format, it required no further conversion.

One-hot Encoding: The key and time_signature features were one-hot encoded. Because key possesses no inherent mathematical order, one-hot encoding prevents the model from assuming a false hierarchy. Although time_signature has a sequential nature, its relationship is non-linear, making one-hot encoding the more appropriate choice.

Label Encoding: The target variable, track_genre, was label encoded. With 114 unique classes, label encoding provides a distinct numerical identifier for each genre, allowing the classifier to treat each class as a separate category without assuming an ordinal relationship between them.

Z-score Scaling: Finally, all continuous numerical features were standardized using Z-score scaling. Features such as duration_ms and tempo exist on large numerical scales, while others like danceability and valence are inherently normalized to a 0–1 scale. Applying standardization across all continuous features shifts their distributions to a mean of 0 and a variance of 1, ensuring that features with larger original magnitudes do not disproportionately influence calculations.

4. Results

4.1. **Data Preprocessing:** This section presents a comprehensive report of the data preprocessing outcomes for the Spotify Tracks Dataset. The preprocessing was designed to transform the raw dataset into a format suitable for machine learning model training while preserving the integrity and interpretability of the high-level audio features.

Final Table for Dataset:

Preprocessing Step	Columns Before	Columns After	Rows After	Key Outcome
Missing Value Removal	21	21	113,999	1 row dropped
Feature Selection (Drop Columns)	21	16	113,999	5 cols removed
Binary Conversion (explicit)	16	16	113,999	bool → int
One-Hot Encoding (key, time_sig.)	16	29	113,999	+13 binary cols
Label Encoding (track_genre)	29	29	113,999	114 classes → 0–113
Z-score Scaling (11 features)	29	29	113,999	μ=0, σ=1
Final Dataset	21	29 features + target	113,999	Ready for modeling

Missing Value Removal:

Inspection revealed exactly one record with missing values across all 21 columns. This row was removed, reducing the dataset from 114,000 to 113,999 rows (a loss of 0.00088%). Imputation was not warranted given the negligible scale of missingness. The class balance across 114 genres remained effectively undisturbed, with the affected genre reducing from 1,000 to 999 samples.

Feature Selection:

Five columns were dropped as non-informative for genre classification:

- Unnamed: 0 — CSV row-index artifact.
- track_id — Unique alphanumeric identifier; no musical signal.
- artists, album_name, track_name — Free-text metadata. Artist identity in particular was excluded to prevent leakage: retaining it would allow models to memorize artist–genre associations rather than learn generalizable audio patterns.

This reduced the feature set from 21 to 16 columns, retaining 11 continuous audio features, 2 binary features, 2 categorical features, and the target label.

Feature Encoding and Scaling:

All remaining features were encoded or scaled as appropriate. The table below summarizes every transformation applied:

Feature	Original Type	Encoding Method	Result	Rationale
explicit	bool	Binary (0/1)	1 int column	Model compatibility
mode	int (0/1)	None (verified)	Unchanged	Already binary
key	int (0–11)	One-hot encoding	12 binary cols	No ordinal relationship between musical keys

Feature	Original Type	Encoding Method	Result	Rationale
time_signature	int (0,1,3,4,5)	One-hot encoding	5 binary cols	Non-linear effect on genre
track_genre	string (114 classes)	Label encoding	int codes 0–113	Multi-class target identifier
11 audio features	float / int, mixed scales	Z-score (StandardScaler)	μ=0, σ=1 each	Equalizes scale for k-NN and other models

A key design decision was fitting StandardScaler exclusively on the training partition, then applying those parameters to the test set. This prevents data leakage using test-set statistics during scaling would inflate reported model performance.

The one-hot encoded reference column was retained (no dummy drop) as tree-based classifiers are not affected by multicollinearity among binary indicators.

Final Preprocessed Dataset:

After all preprocessing steps, the final dataset ready for modeling has the following composition:

- Shape: 113,999 rows × 29 feature columns + 1 target column.
- 11 Z-score scaled continuous features.
- 2 unscaled binary features: explicit mode.
- 12 one-hot encoded key columns (key_0 – key_11).
- 5 one-hot encoded time_signature columns.
- Target: track_genre encoded as integers 0–113 (114 balanced classes, ~999–1,000 samples each).

The dataset contains no missing values, is fully numeric, and is appropriately scaled for both distance-sensitive (k-NN) and tree-based classifiers. The perfectly balanced class distribution means raw accuracy is a meaningful performance metric, with a random-guess baseline of approximately 0.88% (1/114).

References

[1] maharshipandya, "spotify-tracks-dataset," Hugging Face Datasets, 2025. Available: https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset?library=datasets.

[2] Cabansag, I. J., & Ntegeka, P. (2025, May 22). *Prediction of Spotify Chart success using audio and streaming features*. arXiv.org. https://arxiv.org/abs/2508.11632

[3] Du, J. L. (2022). *Spotify between decades: Hit or miss predictor* (Master's thesis, Tilburg University, School of Humanities & Digital Sciences, Department of Cognitive Science & Artificial Intelligence). Tilburg University. https://arno.uvt.nl/show.cgi?fid=160898

[4] S. Jiang, "Predicting Music Popularity: A Machine Learning Approach Using Spotify Data," Proceedings of the 1st International Conference on Modern Logistics and Supply Chain Management. SCITEPRESS - Science and Technology Publications, pp. 324–328, 2024. doi: 10.5220/0013330000004558.

[5] L. Feng, S. Liu, and J. Yao, "Music Genre Classification with Paralleling Recurrent Convolutional Neural Network," 2017, arXiv. doi: 10.48550/ARXIV.1712.08370.

[6] R. Singhal, S. Srivatsan, and P. Panda, "Classification of Music Genres using Feature Selection and Hyperparameter Tuning," JAICN, vol. 4, no. 3, pp. 167–178, Aug. 2022, doi: 10.36548/jaicn.2022.3.003.

[7] M. Fiqri, et al., "Implementation of Supervised Learning Algorithm on Spotify Dataset," *International Journal of Advanced Technology and Information Systems (IJATIS)*, vol. 2, no. 1, 2025. doi: 10.57152/ijatis.v2i1.1123.

[8] Y. Wu, "Predicting Hit Songs Using Spotify Tracks Dataset," *Scholarly Review*, Winter 2025/2026. doi: 10.70121/001c.154965.